
National University of Computer & Emerging Sciences

Chiniot-Faisalabad (CFD) Campus

CL2006 – Operating Systems Lab

Semester Project Manual

Spring 2026

Project Theme

Hospital Patient Triage & Bed Allocator

Process Management · IPC · Memory Allocation · Synchronization



Lab Instructor: Hassan Ahmed • Email: hassan.ahmed@nu.edu.pk

Contents

1	Project Overview	3
1.1	OS Concepts Integrated	3
1.2	CLO–Project Alignment	4
2	Group Formation & Topic Scope	4
2.1	Group Policy	4
2.2	Fixed Project Theme	4
3	System Architecture	5
3.1	High-Level Component Diagram	5
3.2	Bed / Memory Model	5
4	Project Phases & Deliverables	6
4.1	Phase 1 — Environment Setup & Shell Scripting [CLO-1][CLO-4]	6
4.1.1	Deliverable	6
4.1.2	Minimum Requirements	6
4.1.3	Example Script Header	6
4.2	Phase 2 — Process Management, IPC & Scheduling [CLO-2][CLO-4] . .	7
4.2.1	2A: Process Management	7
4.2.2	2B: Inter-Process Communication	7
4.2.3	2C: Priority Scheduling	8
4.3	Phase 3 — Threads & Synchronization [CLO-2][CLO-4]	8
4.3.1	Required Code Header Format	8
4.4	Phase 4 — Memory Management [CLO-3][CLO-4]	10
5	Technical Standards & Coding Guidelines	11
5.1	File & Naming Conventions	11
5.2	Required Data Structures	11
5.3	Compilation & Build Requirements	12
6	Submission Requirements	12
6.1	Required Deliverables	12
6.2	Final Project Report Structure	12
7	Evaluation Rubric	13
7.1	Mark Distribution by Phase	13
7.2	Detailed Rubric	13
8	Individual Contribution Declaration	15
9	Timeline & Deadlines	16
10	Individual Portfolio Requirements (Pre-Viva Eligibility)	16
11	Academic Integrity Policy	17
12	Quick Reference Checklist	18

A Semester Project Self-Evaluation Form

19

1 Project Overview

The Operating Systems Lab Semester Project is a **comprehensive, team-based assessment** that integrates the core OS concepts developed across the lab course into a single cohesive application. Each group of **two students** will design, implement, and demonstrate a fully functional system-level C program that simulates real-world hospital emergency-room operations while concretely exercising process management, inter-process communication, CPU scheduling, synchronization, and memory allocation mechanisms taught in the lab.

Project Theme: Hospital Patient Triage & Bed Allocator

Real-Life Scenario: Patients arrive at an emergency room, are triaged (a priority level is assigned), and are then allocated to an appropriate hospital bed. Beds are limited and come in three types: **ICU**, **General Ward**, and **Isolation**. The system must handle concurrent patient arrivals, enforce priority-based scheduling, prevent race conditions on shared resources, and manage bed memory using the **Best-Fit** allocation strategy.

1.1 OS Concepts Integrated

OS Concept	Project Manifestation	Lab Weeks
Linux & Shell	Makefile build, shell launch scripts, environment setup	1–4
Process Management	<code>fork()</code> / <code>exec()</code> per patient arrival; patient simulator process	5–6
IPC – Pipes	Triage data passed from patient process to admissions via pipes	7
IPC – Shared Memory	Bed allocation bitmap shared between admissions and patient processes	8
CPU Scheduling	Priority queue (triage level) implemented in admissions scheduler	9
Threads	Nurse threads monitor each occupied bed; receptionist thread handles arrivals	10
Mutex / Condition Vars	Protect bed bitmap; signal bed-freed events to waiting patients	11
Semaphores	Limit concurrent ICU/Isolation admissions; producer-consumer patient queue	12
Memory Management	Ward modeled as contiguous block; Best-Fit bed assignment; coalescing	13
Paging / Fragmentation	Track and report external fragmentation across ward partitions	14
Virtual Memory	(Bonus) mmap-based patient record log	15

Table 1: Mapping of OS Lab Topics to Project Components

1.2 CLO–Project Alignment

CLO	Statement	Project Phase
CLO-1	Linux environment, shell scripting, system-level operations	Phase 1
CLO-2	Process/thread management, IPC, synchronization, scheduling	Phase 2 + Phase 3
CLO-3	Memory management and hardware communication simulation	Phase 4
CLO-4	Individual and team professionalism	All Phases

Table 2: Mapping of Course Learning Outcomes to Project Phases

2 Group Formation & Topic Scope

2.1 Group Policy

- Each group consists of **exactly 2 members**. No solo or larger groups permitted.
- Groups must be finalized and communicated to the instructor by **end of Week 13**.
- Both members must contribute to **all phases**. Role-based division is allowed, but **both** members must be able to explain any part of the code during the viva.
- Any form of plagiarism — between groups or from online/AI sources without disclosure — results in **zero marks** for all involved parties.

2.2 Fixed Project Theme

All groups implement the **Hospital Patient Triage & Bed Allocator**. The core architecture is fixed; however, groups may extend the domain (e.g., add a pharmacy module, multi-ward support) with instructor approval for bonus marks.

Important Note

No two groups may produce identical code. Each group must make independent design decisions (e.g., data structures, IPC method variant, scheduling queue implementation). Structural similarity between submissions will be treated as plagiarism.

3 System Architecture

3.1 High-Level Component Diagram

The system comprises the following interacting components:

Core System Components

- **Admissions Manager Process** (`admissions`): The central process. Maintains the bed allocation bitmap in shared memory, runs the priority scheduler, and forks a `patient_simulator` process for each admitted patient.
- **Patient Simulator Process** (`patient_simulator`): Spawned via `fork()` + `execv()` for each patient. Receives patient data via a pipe, “receives treatment” for a randomized duration, then signals the admissions manager to free its bed.
- **Triage Shell Script** (`triage.sh`): Accepts patient name and symptom severity from the command line, computes a triage priority (1–5), and pipes the data to the admissions manager.
- **Nurse Thread Pool**: A configurable pool of POSIX threads within the admissions manager. Each nurse thread monitors one occupied bed using a condition variable; it wakes when a patient’s treatment completes.
- **Shared Memory Bed Bitmap**: A fixed-size shared memory segment representing ward beds. Each cell encodes bed type, status (FREE/OCCUPIED), and patient ID.

3.2 Bed / Memory Model

The hospital ward is modeled as a **contiguous memory block** divided into partitions:

Bed Type	Count	Care Units	Allocation Rule
ICU	4	3	Patients with triage priority 1–2 only
Isolation	4	2	Patients with triage priority 3 or infectious flag
General Ward	12	1	All other patients

Table 3: Ward Configuration (minimum defaults — groups may adjust counts)

Bed assignment uses **Best-Fit**: the smallest available partition (bed) whose care-unit requirement $\geq$ the patient’s required care level is chosen. When a patient is discharged, their bed is freed and **adjacent free beds are coalesced**.

4 Project Phases & Deliverables

The project is divided into **four phases**. Each phase maps directly to lab topics and has specific quantitative requirements.

4.1 Phase 1 — Environment Setup & Shell Scripting [CLO-1][CLO-4]

4.1.1 Deliverable

A set of **shell scripts** and a **Makefile** that bootstrap and manage the entire project environment.

4.1.2 Minimum Requirements

Phase 1 – Minimum Requirements

- *** Triage Script (triage.sh):**
 - Accept patient name, age, and symptom severity (1–10) as command-line arguments.
 - Validate all inputs (non-empty name, numeric age/severity, range checks).
 - Compute triage priority (1–5) from severity using a defined mapping.
 - Output a formatted patient record string and pipe it to `admissions`.
- *** Admissions Launch Script (start_hospital.sh):**
 - Initialize shared memory segment and semaphores.
 - Launch the `admissions` manager process in the background.
 - Print a startup banner showing ward capacity.
- *** Shutdown Script (stop_hospital.sh):**
 - Send `SIGTERM` to the `admissions` process.
 - Clean up shared memory and named semaphores (`ipcrm`, `sem_unlink`).
 - Print a final ward status summary (beds freed, total patients served).
- *** Makefile:**
 - Targets: `all`, `clean`, `run`, `test`.
 - Compile `admissions.c` and `patient_simulator.c` with `-Wall -pthread`.
- **+ Automated stress-test script that spawns 20 patient arrivals in rapid succession.**

4.1.3 Example Script Header

```

1  #!/bin/bash
2  # =====
3  # Project   : Hospital Patient Triage & Bed Allocator
4  # Script    : triage.sh
5  # Group     : Group XX
6  # Members   : Ali Hassan (24F-1234), Sara Khan (24F-5678)
7  # Date      : 2026-04-01
8  # Purpose   : Compute triage priority and pipe patient data
9  #             to the admissions manager process.
10 # Usage     : ./triage.sh <name> <age> <severity 1-10>
11 # =====

```

Listing 1: Required Script Header Format

4.2 Phase 2 — Process Management, IPC & Scheduling [CLO-2][CLO-4]

This phase is the core of the project. All components must be implemented in **C** on Linux.

4.2.1 2A: Process Management

Process Requirements

- ***** The **admissions** manager calls `fork()` for each newly admitted patient.
- ***** The child process calls `execv()` to replace itself with **patient_simulator**, passing patient ID, triage level, and assigned bed ID as arguments.
- ***** **admissions** uses `waitpid()` with `WNOHANG` to reap terminated patient processes without blocking.
- ***** Handle at least **10 concurrent patient processes** simultaneously.
- ***** Implement a **SIGCHLD handler** in **admissions** that automatically reaps zombies.
- ***** **patient_simulator** must:
 - Print arrival, treatment-start, and discharge messages to `stdout`.
 - Sleep for a random duration (simulating treatment): ICU: 5–15 s, General: 2–8 s, Isolation: 3–10 s.
 - On completion, write its patient ID to a named pipe to notify **admissions**.

4.2.2 2B: Inter-Process Communication

IPC Requirements

- ***** **Ordinary Pipe:** `triage.sh` → **admissions**: patient record transmitted via an anonymous pipe on process creation.
- ***** **Named Pipe (FIFO):** **patient_simulator** → **admissions**: discharge notification written to a named FIFO (`/tmp/discharge_fifo`).
- ***** **Shared Memory** (`shmget` / `shmat` or POSIX `shm_open`): Bed allocation bitmap is stored in a shared memory segment, accessible by both **admissions** and all patient processes.
- ***** Implement a **clean shutdown** that detaches and removes the shared memory segment.

4.2.3 2C: Priority Scheduling

Scheduling Requirements

- ***** Implement a **Priority Queue** (min-heap or sorted linked list) inside **admissions** to hold waiting patients when all suitable beds are occupied.
- ***** Patients with triage priority 1 (critical) are always dequeued before priority 2, and so on.
- ***** Implement and demonstrate at least **2 of the following** scheduling simulations (output a Gantt-style log to a file `schedule_log.txt`):
 - First-Come First-Served (FCFS)
 - Shortest Job First (SJF) — treatment time as burst time
 - Priority Scheduling (triage level as priority)
 - Round Robin (RR) with configurable time quantum
- ***** Report average waiting time and average turnaround time for the simulated run.

4.3 Phase 3 — Threads & Synchronization [CLO-2][CLO-4]

Thread & Synchronization Requirements

3.1 POSIX Threads (minimum 3 thread roles)

- ***** **Receptionist Thread**: Continuously reads from the triage FIFO; enqueues new patients into the priority queue.
- ***** **Scheduler Thread**: Dequeues highest-priority patient from the queue; performs Best-Fit bed search; triggers `fork()` + `exec()` for admission.
- ***** **Nurse Threads** (one per bed type, minimum 3): Monitor their bed-type partition; on discharge signal, free the bed, attempt coalescing, and signal the scheduler.

3.2 Mutex & Condition Variables

- ***** A `pthread_mutex_t` must protect **all reads and writes** to the shared bed bitmap.
- ***** A `pthread_cond_t` (`bed_freed`) is broadcast by nurse threads when a bed becomes available; the scheduler thread waits on this condition when no suitable bed exists.
- ***** A separate mutex + condition variable must protect the patient priority queue (producer: receptionist, consumer: scheduler).

3.3 Semaphores

- ***** Use a **counting semaphore** (`sem_t`) to enforce the ICU bed capacity limit (max concurrent ICU admissions = 4).
- ***** Use a **counting semaphore** to enforce Isolation bed limit (max = 4).
- ***** Demonstrate **producer-consumer** synchronization: receptionist (producer) and scheduler (consumer) access the patient queue through a bounded semaphore.
- ***** Demonstrate a scenario where a semaphore **blocks** admission and then **releases** when a patient is discharged.

4.3.1 Required Code Header Format

1 /*

```
2  * =====
3  * Project   : Hospital Patient Triage & Bed Allocator
4  * File      : admissions.c
5  * Group     : Group XX
6  * Members  : Ali Hassan (24F-1234), Sara Khan (24F-5678)
7  * Date      : 2026-04-01
8  * Purpose  : Central admissions manager - process spawning,
9  *            IPC, thread pool, scheduling, and bed allocation.
10 * Compile   : gcc -Wall -o admissions admissions.c -lpthread
11 * =====
12 */
```

Listing 2: Required C Source Header

4.4 Phase 4 — Memory Management [CLO-3][CLO-4]

Memory Management Requirements

4.1 Best-Fit Bed Allocator

- * Model the entire ward as a **contiguous array** (simulated memory block) where each element represents one care unit.
- * Implement **Best-Fit** allocation: scan all free partitions and select the one whose size most closely matches the patient's required care units (without going under).
- * Also implement **First-Fit** and **Worst-Fit** for comparison. Allow the allocation strategy to be selected at runtime via a command-line flag (`--strategy best|first|worst`).
- * Maintain a **free list** of available partitions updated on every allocation and deallocation.

4.2 Coalescing

- * When a patient is discharged, the freed partition is **merged with adjacent free partitions** (left-coalesce and right-coalesce where applicable).
- * Print a before/after ward map to `stdout` showing the coalescing operation.

4.3 Fragmentation Reporting

- * After every allocation/deallocation cycle, compute and print:
 - **Total free units** in the ward.
 - **Largest contiguous free block** size.
 - **External fragmentation percentage:** $(1 - \frac{\text{Largest Free Block}}{\text{Total Free Units}}) \times 100\%$.
- * Log fragmentation statistics to `memory_log.txt` with a timestamp per event.

4.4 Paging Simulation

- * Implement a simple **page table simulation**: divide the ward into fixed-size pages (e.g., page size = 2 care units). Maintain a page table array tracking which patient occupies each page.
- * Report internal fragmentation (wasted units within a page) when a patient's required care units are not a multiple of the page size.

4.5 Bonus — Virtual Memory with mmap

- + Use `mmap()` to memory-map a patient record log file (`patient_records.dat`). Write each patient's admission and discharge record directly to the mapped region. On program exit, `msync()` and `munmap()` the file.

5 Technical Standards & Coding Guidelines

5.1 File & Naming Conventions

Object	Convention	Example
C Source File	lower_snake_case.c	admissions.c
Shell Script	verb_noun.sh	start_hospital.sh
Header File	lower_snake_case.h	bed_allocator.h
Shared Memory Key	Hex constant defined in header	0xBEDFOOD
Named Semaphore	/sem_noun	/sem_icu_limit
Named FIFO	/tmp/noun_fifo	/tmp/discharge_fifo
Struct	PascalCase	PatientRecord, BedPartition
Function	verb_noun()	allocate_bed(), coalesce_free()
Macro / Constant	ALL_CAPS	MAX_BEDS, ICU_CAPACITY

Table 4: Naming Conventions

5.2 Required Data Structures

Your submission must define and use (at minimum) the following structs in a shared header:

```

1  /* Patient record passed via IPC */
2  typedef struct {
3      int    patient_id;
4      char   name[64];
5      int    age;
6      int    severity;          /* 1-10 raw severity from triage */
7      int    priority;          /* 1-5 computed triage priority */
8      int    care_units;        /* memory units required */
9      time_t arrival_time;
10 } PatientRecord;
11
12 /* Single bed partition in the ward memory model */
13 typedef struct {
14     int    partition_id;
15     int    start_unit;          /* index in ward array */
16     int    size;                /* number of care units */
17     int    is_free;             /* 1 = FREE, 0 = OCCUPIED */
18     int    patient_id;          /* -1 if free */
19     char   bed_type[16];        /* "ICU", "GENERAL", "ISOLATION" */
20 } BedPartition;

```

Listing 3: Minimum Required Struct Definitions

5.3 Compilation & Build Requirements

- All C code must compile with **zero warnings** under `gcc -Wall -Wextra -pthread`.
- A single `make all` from the project root must build the entire system.
- A single `make run` must start the hospital simulation.
- Use `valgrind` to check for memory leaks and include a screenshot of the `valgrind` output in the report.

6 Submission Requirements

6.1 Required Deliverables

Submit a single compressed folder named:

`24F-XXXX_24F-YYYY_OS_Lab_Project.zip`

Example: `24F-1234_24F-5678_OS_Lab_Project.zip`

The folder must contain:

#	File / Folder	Contents
1	<code>src/</code>	All <code>.c</code> and <code>.h</code> source files
2	<code>scripts/</code>	All shell scripts (<code>.sh</code>)
3	<code>Makefile</code>	Build system (<code>all</code> , <code>clean</code> , <code>run</code> , <code>test</code>)
4	<code>README.md</code>	Setup instructions, dependencies, run guide
5	<code>logs/</code>	Sample <code>schedule_log.txt</code> and <code>memory_log.txt</code>
6	<code>report.pdf</code>	Final project report (see Section 6.2)
7	<code>demo_video.mp4</code>	5–10 minute walkthrough with voice-over
8	<code>self_eval.pdf</code>	Completed and signed self-evaluation form

Table 5: Required Submission Files

6.2 Final Project Report Structure

The report (`report.pdf`) must include:

1. **Title Page:** Project name, group members, student IDs, date.
2. **Table of Contents.**
3. **Introduction:** Problem statement, objectives, and system overview.
4. **System Design:** Architecture diagram, component descriptions, data flow.
5. **Phase 1 Documentation:** Shell scripts explained with sample runs.
6. **Phase 2 Documentation:** Process management code walkthrough, IPC design, scheduling output with Gantt chart, average waiting/turnaround times.
7. **Phase 3 Documentation:** Thread design, mutex/condition variable usage, semaphore scenarios (include a scenario showing a blocked and released admission).

8. **Phase 4 Documentation:** Memory layout diagram, Best-Fit trace (step-by-step example), fragmentation statistics table, paging simulation output.
9. **Testing:** Test cases for each phase with expected vs. actual output screenshots.
10. **Valgrind Output:** Screenshot confirming zero memory leaks.
11. **Challenges & Lessons Learned.**
12. **Individual Contribution Table** (see Section 7).

7 Evaluation Rubric

Total: 100 Marks (contributes 20% of the final course grade)

7.1 Mark Distribution by Phase

Phase	Component	Marks	CLO
1	Shell Scripting & Makefile	15	CLO-1
2A	Process Management (<code>fork/exec/signals</code>)	15	CLO-2
2B	Inter-Process Communication (Pipes, Shared Memory)	10	CLO-2
2C	CPU Scheduling Simulation	10	CLO-2
3	Threads, Mutex, Condition Variables, Semaphores	20	CLO-2
4	Memory Allocation, Coalescing, Fragmentation, Paging	20	CLO-3
–	Report, Demo Video & Code Quality	5	CLO-4
–	Viva / Presentation	5	CLO-4
Total		100	

Table 6: Mark Distribution Across Phases

7.2 Detailed Rubric

<code>triage.sh</code> with correct argument validation and priority mapping	4
<code>start_hospital.sh</code> initializes shared memory and launches admissions	3
<code>stop_hospital.sh</code> cleans up all IPC resources and prints summary	3
Makefile with all four targets compiling without warnings	3
Code comments, script headers, and portability	2
Correct <code>fork()</code> + <code>execv()</code> for each patient admission	5
<code>SIGCHLD</code> handler + <code>waitpid(WNOHANG)</code> zombie reaping	4
<code>patient_simulator</code> prints lifecycle messages and sleeps correctly	3
10+ concurrent processes handled without crashes	3

Anonymous pipe correctly transmits patient record from triage to admissions	3
Named FIFO correctly signals discharge from patient process to admissions	3
Shared memory segment created, attached, detached, and removed cleanly	4

Priority queue implemented and correct order of admission verified	4
2+ scheduling algorithms simulated with log output	4
Correct average waiting and turnaround time computation	2

3 thread roles implemented (receptionist, scheduler, nurse pool)	6
Mutex correctly protects bed bitmap (no data races detected)	4
Condition variable correctly used for bed-freed notification	4
Semaphores enforce ICU/Isolation capacity limits correctly	4
Producer-consumer pattern demonstrated with bounded semaphore	2

Best-Fit allocator correctly selects partition and updates free list	6
First-Fit and Worst-Fit implemented; runtime strategy selection works	3
Coalescing merges adjacent free partitions correctly	4
External fragmentation percentage computed and logged correctly	3
Paging simulation reports internal fragmentation per patient	3
Bonus: <code>mmap</code> -based patient record log	+3

Report complete with all required sections and screenshots	2
Demo video: voice-over explains all four phases with live demo	2
Code: consistent style, comments, zero compiler warnings	1

Both members explain all phases clearly	2
Live demonstration runs without crashes	2
Ability to answer cross-questions on OS concepts	1

8 Individual Contribution Declaration

Both members must include a **signed contribution table** in the final report. Viva marks are awarded **individually** based on demonstrated understanding.

Phase	Member 1 Contribution	Member 2 Contribution
Phase 1: Shell Scripts		
Phase 1: Makefile		
Phase 2A: Process Management		
Phase 2B: IPC		
Phase 2C: Scheduling		
Phase 3: Threads		
Phase 3: Mutex / CondVars		
Phase 3: Semaphores		
Phase 4: Memory Allocator		
Phase 4: Fragmentation / Paging		
Report & Video		
Signatures		

Table 7: Individual Contribution Declaration Table

Important Note

If a member cannot explain a line of code attributed to them during the viva, marks will be deducted at the instructor's discretion.

9 Timeline & Deadlines

Week	Milestone	Deadline	Action Required
13	Group Formation	End of Week 13	Submit group names to instructor
13	Phase 1	End of Week 13	Submit all shell scripts + Makefile
14	Phase 2A + 2B	End of Week 14	Submit process management + IPC code
14	Phase 2C	End of Week 14	Submit scheduling simulation + log output
15	Phase 3	End of Week 15	Submit threads + synchronization code
16	Phase 4 + Full Submission	End of Week 16	Submit complete project .zip
Dead Week	Viva / Presentation	Dead Week	In-lab live demonstration

Table 8: Project Timeline

Important Note

Late Submission Policy: A deduction of **25% per day** applies to any late submission. No submission accepted more than 3 days past the deadline without prior written approval from the instructor.

10 Individual Portfolio Requirements (Pre-Viva Eligibility)

Individual Requirement — Both Members Must Complete

This section is **individual**. Every student must complete all three tasks independently to be eligible for the final Viva.

Required Actions per Student:

-  **GitHub Repository:**
 - Create a **public repository** and upload your project source.
 - Write a `README.md` covering: project description, build instructions, how to run, OS concepts demonstrated, and your personal contribution.
 - Use meaningful commit messages — do not push everything in a single commit.
-  **LinkedIn Post:**

- Share a terminal screenshot or short clip of your system running (e.g., patients being admitted, bed bitmap being updated).
- Write about one technical challenge you solved (e.g., *“Debugging a race condition on the bed bitmap when two threads tried to allocate the last ICU bed simultaneously...”*).
- Tag your teammate and mention your instructor.

3. Medium Blog Article:

- Write a technical post of **300–500 words**.
- Suggested topics: how Best-Fit compares to First-Fit in your simulation, why you chose condition variables over busy-waiting, lessons from debugging `fork()` + `exec()` interactions, or how external fragmentation emerged in your ward.

Verification Process

- A shared Google Sheet is linked on Google Classroom.
- Every student must paste their **GitHub, LinkedIn, and Medium URLs** in the respective columns next to their name.
- Sheet URL: [Click here to access the Verification Sheet](#)

Failure to populate the Google Sheet with valid links before the viva renders the student **ineligible for the Viva Voce**, resulting in an **automatic Zero (0) for the entire project component**.

11 Academic Integrity Policy

1. All work must be original and created by the group members.
2. Sharing source code between groups is strictly prohibited, even “for reference.”
3. Use of AI code generation tools (ChatGPT, Claude, Copilot, Gemini, etc.) to produce entire functions, scripts, or modules without understanding them is considered plagiarism.
4. During the viva, inability to explain any part of your own code will be treated as academic dishonesty.
5. Automated similarity checks are run on all submissions. Confirmed similarity between any two groups results in **zero marks for all involved**.
6. External libraries (beyond standard POSIX/glibc) must be cited in the README and report.

12 Quick Reference Checklist

Use this checklist before final submission.

Requirement	Min. Count
<code>triage.sh</code> with input validation and priority mapping	1
<code>start.hospital.sh</code> initializes IPC resources	1
<code>stop.hospital.sh</code> cleans up all IPC resources	1
Makefile targets: <code>all</code> , <code>clean</code> , <code>run</code> , <code>test</code>	4 targets
<code>fork()</code> + <code>execv()</code> per patient admission	1 per patient
SIGCHLD handler + <code>waitpid(WNOHANG)</code>	1
<code>patient.simulator</code> prints lifecycle + sleeps per bed type	1
10+ concurrent patient processes handled	10
Anonymous pipe: <code>triage</code> → <code>admissions</code>	1
Named FIFO: <code>discharge</code> notification	1
Shared memory segment for bed bitmap	1
Priority queue for waiting patients	1
Scheduling algorithms simulated with log	2 algorithms
Average waiting and turnaround time reported	1
Receptionist, scheduler, nurse POSIX threads	3 roles
Mutex protecting bed bitmap	1
Condition variable (<code>bed.freed</code>) used	1
Semaphore enforcing ICU capacity	1
Semaphore enforcing Isolation capacity	1
Producer-consumer via bounded semaphore	1
Best-Fit allocator with free list	1
First-Fit and Worst-Fit implemented	2
Runtime strategy selection (<code>--strategy</code>)	1
Coalescing of adjacent free partitions	1
External fragmentation percentage computed and logged	1
Paging simulation with internal fragmentation report	1
Valgrind screenshot (zero leaks) in report	1
Report: all required sections with screenshots	12 sections
Demo video with voice-over (5–10 min)	1
GitHub, LinkedIn, Medium links in Google Sheet	Both members
Contribution table signed by both members	1

Table 9: Pre-Submission Checklist

A Semester Project Self-Evaluation Form

Instructions for Completing This Form

Complete this form **honestly** before the Viva. Submit it as part of your **.zip** folder (scanned/signed PDF). The instructor uses this to calibrate Viva questions to what you claim to have implemented. Inflated self-evaluations that do not match the demo result in mark deductions.

A. Group Details

Section: BSCS

Session: Spring 2026

Lab Instructor: Hassan Ahmed

#	Student Name	Roll Number
1.	_____	_____
2.	_____	_____

Group Member Details

GitHub Repository URL: _____

B. Module Implementation Checklist

Check the box **only** if the module is **fully functional**. Partial or crash-prone implementations should be left unchecked and described in Section D.

I. Shell Scripting & Environment

triage.sh: Input validation, priority mapping, correct pipe output.

start_hospital.sh: IPC initialization, admissions process launched.

stop_hospital.sh: Clean shutdown, IPC cleanup, summary printed.

Makefile: All four targets work; compiles with zero warnings.

II. Process Management & IPC

fork() + execv(): New process spawned per patient admission.

SIGCHLD + waitpid(WNOHANG): Zombie processes reaped automatically.

patient_simulator: Prints lifecycle messages; sleeps per bed type.

10+ concurrent processes: System remains stable under load.

Anonymous Pipe: Patient record transmitted from triage to admissions.

Named FIFO: Discharge notification from patient process to admissions.

Shared Memory: Bed bitmap in shared segment; clean detach/removal.

III. Scheduling

Priority Queue: Patients dequeued in triage-priority order.

2+ Scheduling Algorithms: Simulated with log output in `schedule_log.txt`.

Metrics: Average waiting time and turnaround time reported correctly.

IV. Threads & Synchronization

Receptionist Thread: Reads from FIFO; enqueues patients.

Scheduler Thread: Dequeues, selects bed, triggers fork/exec.

Nurse Threads (3+): Monitor bed types; trigger coalescing on discharge.

Mutex: All bitmap accesses protected; no data races.

Condition Variable (bed_freed): Scheduler waits; nurse broadcasts.

ICU Semaphore: Blocks admission when ICU full; releases on discharge.

Isolation Semaphore: Enforces isolation bed limit correctly.

Producer-Consumer: Bounded semaphore demonstrated between receptionist and scheduler.

V. Memory Management

Best-Fit Allocator: Correct partition selected; free list updated.

First-Fit & Worst-Fit: Implemented; runtime strategy flag works.

Coalescing: Adjacent free partitions merged after discharge.

Fragmentation Report: External fragmentation % computed and logged.

Paging Simulation: Internal fragmentation reported per patient.

Valgrind: Zero memory leaks confirmed; screenshot in report.

VI. Bonus Features (Optional)

mmap-based patient record log (Phase 4 bonus).

Automated stress-test script (20 rapid arrivals).

Role-based dashboard: Admin sees all wards; Nurse sees own bed type.

C. Individual Contributions

Identify who built what. Vague entries like “I helped” are not acceptable.

Member 1: _____ Roll No.: _____

Specific modules/tasks implemented:

Member 2: _____ Roll No.: _____

Specific modules/tasks implemented:

D. Reflection & Analysis

1. The “Synchronization” Challenge:

(Did you encounter a race condition or deadlock? How did you detect and fix it?)

2. Memory Allocation Trade-offs:

(Did Best-Fit always outperform First-Fit in your tests? Were there cases where it performed worse? Why?)

3. Known Bugs / Unfinished Modules:

(Be honest. Does the system crash on a specific edge case? Is coalescing incomplete?)

E. Individual Portfolio Links

Platform	Member 1 URL	Member 2 URL
 GitHub		
 LinkedIn		
 Medium Blog		

Portfolio Links (must also be entered in the shared Google Sheet)

F. Signatures & Integrity Declaration

By signing below, we certify that **all work in this project was created by us**. We understand that code generated entirely by AI without understanding, or copied from any online source or another group, constitutes academic dishonesty and will result in **disqualification from the Viva and zero marks** for the entire project.

Member 1 Signature

Name: _____

Date: _____

Member 2 Signature

Name: _____

Date: _____